Assignment

Bachelor of Computer Applications

Semester III

Database Management System

Topic: RELATIONAL DATABASE VS. NoSQL DATABASE

BY

Name: M Blessy

Registration Number: 2511021240067

DEPARTMENT OF COMPUTER APPLICATIONS

Alliance university

Chikkahagade cross, Anekal, Bangaluru 562106

RELATIONAL DATABASE VS. NoSQL DATABASE

1. Introduction

A database is an organized collection of data that allows users to store, retrieve, update, and manage information efficiently. Organizations use different database technologies depending on their data structure, business requirements, performance needs, and scalability requirements.

The two major categories of databases are relational databases and NoSQL databases.

Relational databases store data in tables containing rows and columns. NoSQL databases use different data models, such as documents, key-value pairs, graphs, and wide-column structures.

Both technologies are useful, but each is suitable for different types of applications.

2. Relational Database

A relational database stores data in tables. Each table represents an entity, and relationships between tables are established using primary keys and foreign keys.

Relational databases generally use Structured Query Language (SQL) to create tables, insert records, retrieve information, and update data.

Main features:

- Data is organized into tables with rows and columns.
- Primary keys identify records uniquely.
- Foreign keys establish relationships between tables.
- SQL is used to manage and retrieve data.
- Transactions commonly follow ACID properties: Atomicity, Consistency, Isolation, and Durability.
- The schema is generally defined before data is inserted.

Examples: MySQL, PostgreSQL, Oracle Database, and Microsoft SQL Server.

Real-world example: A hospital can use MySQL to manage patients, doctors, appointments, admissions, and bills. A patient's ID can connect their appointment and billing records, helping maintain accurate relationships between the data.

3. NoSQL Database

NoSQL stands for “Not Only SQL.” NoSQL databases are designed to handle different kinds of data and flexible data structures. Depending on the database type, data may be stored as documents, key-value pairs, graphs, or wide-column records.

NoSQL databases are often used in applications that require flexible schemas, distributed data storage, or large-scale data processing.

Main features:

- Supports flexible or varied data structures, depending on the database type.
- Can store structured, semi-structured, and some unstructured data.
- Many NoSQL systems support horizontal scaling across multiple servers.
- Data models can be selected according to application requirements.
- Consistency and transaction guarantees vary between products and configurations.
- Some systems are designed for high-volume reads and writes across distributed environments.

Examples: MongoDB, Apache Cassandra, Redis, and Neo4j.

Real-world example: An online shopping application can use MongoDB to store product information when different products have different attributes. A mobile phone may have RAM and storage details, while clothing may have size and colour details. A flexible document structure can accommodate these differences.

4. Comparison Between Relational and NoSQL Databases

Basis	Relational Database	NoSQL Database
Data structure	Tables with rows and columns	Documents, key-value pairs, graphs, or wide-column records
Schema	Generally predefined	Often flexible, depending on the system
Query language	Commonly uses SQL	Uses database-specific query methods or APIs; some systems also support SQL-like interfaces
Scalability	Can scale vertically and also horizontally in supported systems	Many systems are designed for horizontal scaling
Consistency	Commonly provides strong transactional consistency through ACID transactions	Consistency guarantees vary; some systems provide strong consistency and others offer configurable or eventual consistency
Performance	Effective for structured data, joins, and complex queries	Can be highly efficient for particular access patterns and large distributed workloads
Relationships	Supports relationships using primary keys, foreign keys, and joins	Relationships may be represented through embedded documents, references, or graph connections
Transactions	Strong multi-record transactions are widely supported	Transaction support varies by product; some support multi-record transactions
Data flexibility	Changes to the schema may require planned modifications	Flexible models can make changing data structures easier

Best suited for	Structured data and applications requiring reliable relationships and transactions	Applications requiring flexible data models or particular distributed data-access patterns
Examples	MySQL, PostgreSQL, Oracle Database	MongoDB, Cassandra, Redis, Neo4j

5. Comparison Based on Scalability

Scalability refers to the ability of a database to handle an increase in users, data, or workload.

Relational Database Scalability

Relational databases can scale vertically by increasing the CPU, RAM, and storage of a server. Some relational systems also support horizontal scaling through replication, partitioning, sharding, or distributed architectures.

They are suitable for applications requiring structured data and complex relationships, although scaling distributed relational workloads may require careful design.

Example: A hospital can upgrade its MySQL server as patient records and appointment transactions increase.

NoSQL Database Scalability

Many NoSQL databases are designed to distribute data across multiple servers. This is known as horizontal scaling. It allows an organization to add servers to handle increasing workloads.

The actual scalability depends on the database product, data model, partitioning strategy, and workload.

Example: A large online shopping platform may distribute product catalog data across multiple servers to support a growing number of products and users.

6. Comparison Based on Consistency

Consistency means ensuring that data follows the database's rules and that users receive results according to the system's consistency guarantees.

Relational Database Consistency

Relational databases commonly support ACID transactions. These properties help ensure that transactions are completed reliably and that database rules are maintained.

For example, when a hospital records a bill payment, the system should reliably update the payment record without leaving the transaction in an incomplete state.

NoSQL Database Consistency

NoSQL consistency depends on the particular database. Some NoSQL databases provide strong consistency, while others allow eventual consistency or configurable consistency levels.

Eventual consistency means that updates may not be visible on every replica immediately, but replicas are expected to converge if updates stop and the system continues operating normally.

For example, a distributed product catalog may tolerate a short delay before a product update appears on every server. However, a payment system may require stronger guarantees for financial transactions.

Therefore, it is incorrect to say that every relational database is strongly consistent or that every NoSQL database is eventually consistent. The actual guarantees depend on the technology and configuration.

7. Comparison Based on Performance

Performance refers to how quickly a database can process queries, insert records, update information, and return results.

Relational Database Performance

Relational databases perform well when an application needs structured queries, joins, aggregations, and transactions across related tables. Proper indexing and query optimization can improve performance.

Example: A hospital management system can join the Patient, Appointment, and Doctor tables to display appointment details.

NoSQL Database Performance

NoSQL databases can provide high performance when their data model matches the application's access patterns. Some are optimized for key-based lookups, large distributed writes, caching, or graph traversal.

Example: Redis can store frequently accessed session information in memory, allowing applications to retrieve it quickly.

Performance is not determined by the database category alone. Data size, indexing, hardware, query design, and workload all affect the result.

8. Real-World Examples

Example 1: Banking System — Relational Database

A banking system stores customer accounts, transactions, balances, and payment records. These records are closely related, and transactions must be processed reliably.

A relational database such as PostgreSQL or Oracle Database can support structured records, constraints, and ACID transactions.

Reason for choosing it: Reliable transactions, data integrity, and relationships between accounts and transactions.

Example 2: Online Shopping — NoSQL Database

An online shopping platform may manage millions of products with different attributes, customer activity, and rapidly changing catalog information.

A document database such as MongoDB can store product records with flexible structures. Other database technologies may be used for orders, payments, caching, or analytics.

Reason for choosing it: Flexible product data and the ability to scale suitable workloads.

Example 3: Social Networking — NoSQL Database

A social networking application manages users, posts, reactions, messages, and connections. Some of these workloads involve huge volumes of data and users distributed across different locations.

A system such as Apache Cassandra can support distributed, high-volume data workloads. A graph database such as Neo4j can be useful when analysing relationships between users.

Reason for choosing it: Distributed data storage and efficient handling of particular large-scale workloads or relationships.

Example 4: Hospital Management System — Relational Database

The Hospital Management System developed for this project uses MySQL. It contains separate tables for patients, doctors, departments, appointments, admissions, medicines, prescriptions, laboratory tests, rooms, and bills.

Foreign keys connect related records. For example, the Appointment table refers to patients and doctors through their IDs.

Reason for choosing it: The system contains structured records, clearly defined relationships, and transactions that benefit from relational integrity.

9. Why Organizations Choose Different Database Technologies

Organizations select databases according to their technical and business requirements.

1. **Nature of data:** Structured and strongly related data may suit a relational database, while flexible document data may suit a document database.
2. **Scalability requirements:** Applications expecting rapid growth may choose technologies designed for distributed storage and horizontal scaling.
3. **Consistency requirements:** Financial and transaction-heavy systems may prioritize strong transaction guarantees. Other workloads may accept configurable or eventual consistency.
4. **Performance needs:** Applications choose database models that efficiently support their most frequent queries and operations.
5. **Schema flexibility:** Applications with changing or varied data structures may benefit from a flexible data model.
6. **Cost and maintenance:** Organizations consider licensing, infrastructure, staff expertise, backup, monitoring, and operational complexity.
7. **Integration:** The selected database must work with the organization's programming languages, applications, analytics tools, and existing systems.

Many organizations use both relational and NoSQL databases. This approach is known as polyglot persistence. For example, an online shopping application might use a relational database for orders and payments, a document database for product information, and an in-memory database for caching.

The objective is not to choose one technology for every situation, but to choose the technology that best meets each requirement.

10. Conclusion

Relational and NoSQL databases both play important roles in modern information systems.

Relational databases such as MySQL and PostgreSQL are suitable for structured data, well-defined relationships, and applications requiring reliable transactions. NoSQL databases such as MongoDB, Cassandra, Redis, and Neo4j support different data models and can be effective for flexible or distributed workloads.

The choice depends on scalability, consistency, performance, data structure, cost, and application requirements. Some organizations use both types to take advantage of their different strengths.

For the Hospital Management System project, MySQL is an appropriate choice because patient, doctor, appointment, admission, prescription, and billing records have clear relationships that can be managed using tables, keys, constraints, and transactions.